



COURSE PROJECT

Discover Saudi Arabia

An Interactive Cultural Website

اكتشف السعودية، موقع ثقافي تفاعلي

Course	Web Technologies, CSC 457
Instructor	Dr. Sultan Alfarhood
Student	Abdullah Alshammari, 443100803
Semester	Second Semester, Academic Year 1447 H, 2025/2026

2. Project Information

2.1 Project Description

Discover Saudi Arabia is a dynamic, fully bilingual-ready cultural website built with HTML, CSS, JavaScript, and PHP. Its purpose is to introduce visitors to the regions, landmarks, and culture of the Kingdom of Saudi Arabia through an engaging, image-led interface. The public side of the site presents a home page, a regions gallery with live filtering, and a detailed page for each region. The admin side, protected by PHP sessions, offers complete content management through full CRUD operations on the regions database. The project demonstrates the integration of client-side interactivity (night mode, filtering, form validation) with server-side logic (authentication, data persistence, file uploads), all backed by a MySQL database.

2.2 Live Website and Admin Demo Credentials

The credentials below are intended for grading and testing only. They allow access to the protected Admin Dashboard and full CRUD functionality.

Live Website URL	<code>https://capstone.alshammarii.me</code>
Admin Username	<code>demo</code>
Admin Password	<code>demo2026</code>
Login Path	<code>/admin/login.php</code>

2.3 Team Contributions

This project was completed individually. All design, implementation, testing, and documentation work was carried out by the student listed on the cover page.

- Abdullah Alshammari (443100803): Database schema design, PHP server-side logic (login, sessions, full CRUD on the regions table), client-side HTML, CSS theming with night-mode support, JavaScript filtering and form validation, deployment, testing, and final report.

3. Database Schema

3.1 Entity Relationship Diagram (ERD)

The deployed database, hosted as if0_41034301_ksa, contains two tables. The admins table stores the credentials used to access the protected dashboard, while the regions table stores all cultural content displayed on the public side of the website. The admins table is conceptually linked to the regions table because admin users manage region records, but no foreign-key relationship exists since region records are not owned by a specific admin in this implementation. All textual columns use the utf8mb4_unicode_ci collation to support Arabic content correctly.

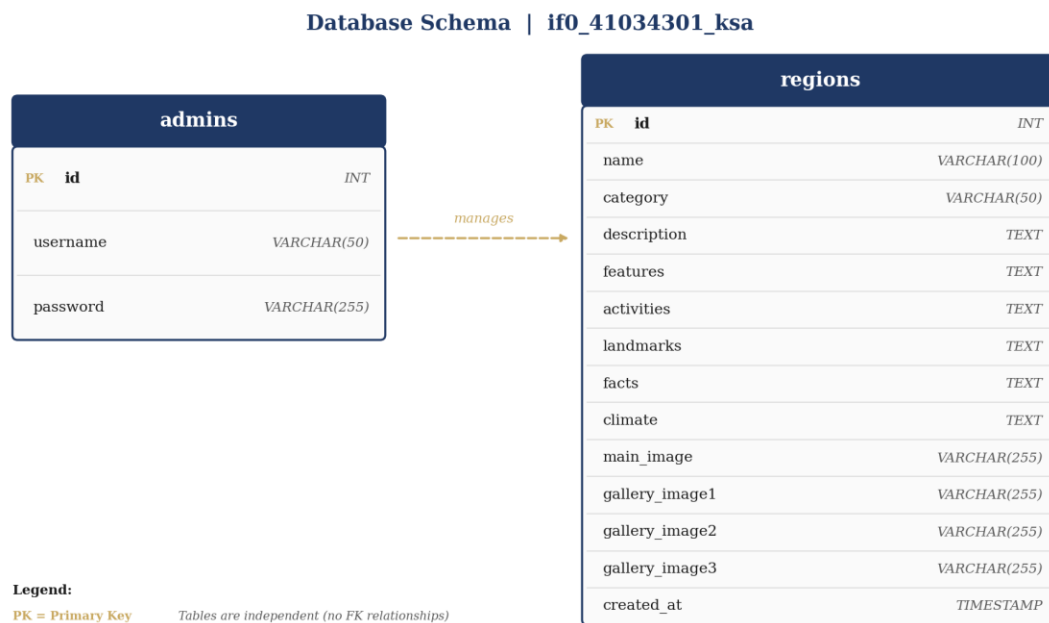


Figure 3.1: ERD of the deployed database (if0_41034301_ksa).

3.2 Database Tables (phpMyAdmin Screenshots)

The database is deployed on the production server and managed through phpMyAdmin. The following screenshots show the live tables and their contents as they appear after deployment, confirming that both tables are present, correctly configured with the utf8mb4_unicode_ci collation (required for Arabic text), and populated with real data.

Table	Action	Rows	Type	Collation
☐ admins		1	InnoDB	utf8mb4_unicode_ci
☐ regions		4	InnoDB	utf8mb4_unicode_ci
2 tables	Sum	5	MyISAM	latin1_swedish_ci

Figure 3.2: Overview of the two tables in the deployed database (admins and regions), shown in phpMyAdmin.

+ Options

	id	username	password
☐  Edit  Copy  Delete	1	admin	\$2y\$10\$XP6smYh8gvq\$PvMKkmBvVu/MuoPzWx06BQNNpNY9yWi...


☐ Check all
 With selected:
  Edit
  Copy
  Delete
  Export

[illegible]

4. Backend Code Samples

The following screenshots highlight the core server-side functionality of the website. Each snippet illustrates a different part of the request lifecycle, from authentication through full CRUD on the regions table. All database access is performed through PDO with prepared statements, which protects the application from SQL injection.

4.1 Admin Login and Session Start

The login script first checks whether a session already exists, in which case the visitor is redirected straight to the dashboard. On a POST request, the submitted credentials are validated, the username is looked up in the admins table using a parameterised SELECT, and the supplied password is checked against the stored bcrypt hash with `password_verify()`. On success, the admin id and username are stored in `$_SESSION` and the user is redirected to the dashboard; on failure, an Arabic error message is shown.

```
2  session_start();
3  require_once '../config/db.php';
4
5  if (isset($_SESSION['admin_id'])) {
6      header("Location: dashboard.php");
7      exit();
8  }
9
10 $error = '';
11 if ($_SERVER['REQUEST_METHOD'] === 'POST') {
12     $username = trim($_POST['username'] ?? '');
13     $password = trim($_POST['password'] ?? '');
14
15     if ($username !== '' && $password !== '') {
16         $stmt = $conn->prepare("SELECT * FROM admins WHERE username = ?");
17         $stmt->execute([$username]);
18         $admin = $stmt->fetch(PDO::FETCH_ASSOC);
19
20         if ($admin && password_verify($password, $admin['password'])) {
21             $_SESSION['admin_id'] = $admin['id'];
22             $_SESSION['admin_username'] = $admin['username'];
23             header("Location: dashboard.php");
24             exit();
25         } else {
26             $error = 'اسم المستخدم أو كلمة المرور غير صحيحة';
27         }
28     } else {
29         $error = 'يرجى ملء جميع الحقول';
30     }
31 }
```

Figure 4.1: login.php, validating the admin credentials with `password_verify()` and starting the session.

4.2 Adding a New Region (CREATE)

The add-content script is protected by auth.php, which redirects unauthenticated visitors back to the login page. On a POST request the script reads each form field, handles the uploaded main image (and likewise the three optional gallery images) by giving them unique filenames and moving them into uploads/images/, and finally inserts a new record into the regions table with a

parameterised INSERT statement. A flash message is stored in the session and the user is redirected to the dashboard, where the success banner is displayed.

```
2  require_once 'auth.php';
3  require_once '../config/db.php';
4
5  if ($_SERVER['REQUEST_METHOD'] === 'POST') {
6      $name = trim($_POST['name'] ?? '');
7      $category = trim($_POST['category'] ?? '');
8      $description = trim($_POST['description'] ?? '');
9      $facts = trim($_POST['facts'] ?? '');
10     $climate = trim($_POST['climate'] ?? '');
11     $activities = trim($_POST['activities'] ?? '');
12     $landmarks = trim($_POST['landmarks'] ?? '');
13
14     $main_image = '';
15     if (isset($_FILES['main_image']) && $_FILES['main_image']['error'] === 0) {
16         $ext = pathinfo($_FILES['main_image']['name'], PATHINFO_EXTENSION);
17         $main_image = 'main_' . time() . '.' . $ext;
18         move_uploaded_file($_FILES['main_image']['tmp_name'], '../uploads/images/' . $main_image);
19     }
20
21     // Likewise $gallery1 , $gallery2 and $gallery 3
22
23     $stmt = $conn->prepare("INSERT INTO regions (name, category, description, facts, climate, acti
24     $stmt->execute([$name, $category, $description, $facts, $climate, $activities, $landmarks, $ma
25
26     $_SESSION['message'] = 'تم إضافة السجل بنجاح';
27     $_SESSION['msg_type'] = 'success';
28     header("Location: dashboard.php");
29     exit();
30 }
```

Figure 4.2: add.php, handling file uploads and inserting a new region into the regions table.

4.3 Retrieving Regions for the Dashboard (READ)

The dashboard issues a single SELECT query against the regions table, ordered by id, and renders each row as an entry in the management table. The same flash-message mechanism shown here is used for every CRUD operation: the script reads any pending message from \$_SESSION, displays it once at the top of the dashboard, and then unsets it so that it does not reappear on subsequent visits.

```
2  require_once 'auth.php';
3  require_once '../config/db.php';
4
5  $stmt = $conn->query("SELECT * FROM regions ORDER BY id ASC");
6  $regions = $stmt->fetchAll(PDO::FETCH_ASSOC);
7
8  $message = '';
9  $msg_type = '';
10 if (isset($_SESSION['message'])) {
11     $message = $_SESSION['message'];
12     $msg_type = $_SESSION['msg_type'] ?? 'success';
13     unset($_SESSION['message']);
14     unset($_SESSION['msg_type']);
15 }
```

Figure 4.3: dashboard.php, fetching all regions and rendering any pending flash message.

4.4 Updating an Existing Region (UPDATE)

The update script first verifies that an id has been supplied, then loads the existing record so that the form can be pre-filled with its current values. On submission, the same field-handling logic used for adding new content is applied: the existing image filename is kept by default, but if a new file is uploaded it replaces the old one. Finally, the record is saved back to the regions table with a parameterised UPDATE statement, and the user is redirected to the dashboard with a success message.

```
2  require_once 'auth.php';
3  require_once '../config/db.php';
4
5  if (!isset($_GET['id']) && !isset($_POST['id'])) {
6      header("Location: dashboard.php");
7      exit();
8  }
9
10 $id = $_GET['id'] ?? $_POST['id'];
11
12 if ($_SERVER['REQUEST_METHOD'] === 'POST') {
13     $name = trim($_POST['name'] ?? '');
14     $category = trim($_POST['category'] ?? '');
15     $description = trim($_POST['description'] ?? '');
16     $facts = trim($_POST['facts'] ?? '');
17     $climate = trim($_POST['climate'] ?? '');
18     $activities = trim($_POST['activities'] ?? '');
19     $landmarks = trim($_POST['landmarks'] ?? '');
20
21     $stmt = $conn->prepare("SELECT * FROM regions WHERE id = ?");
22     $stmt->execute([$id]);
23     $old = $stmt->fetch(PDO::FETCH_ASSOC);
24
25     $main_image = $old['main_image'];
26     if (isset($_FILES['main_image']) && $_FILES['main_image']['error'] === 0) {
27         $ext = pathinfo($_FILES['main_image']['name'], PATHINFO_EXTENSION);
28         $main_image = 'main_' . time() . '.' . $ext;
29         move_uploaded_file($_FILES['main_image']['tmp_name'], '../uploads/images/' . $main_image);
30     }
```

Figure 4.4: edit.php, loading the existing record and processing the update form submission.

4.5 Deleting a Region (DELETE)

The delete script validates that the supplied id is numeric before doing anything else. It then loads the record so that the associated image files (main image plus up to three gallery images) can be removed from disk with `unlink()`, keeping the uploads directory clean. Only after the files have been removed does the script execute a parameterised DELETE against the regions table. The result is reported back to the admin through the same flash-message mechanism used by the other operations. Note that on the dashboard side a JavaScript confirmation dialog is shown before this script is ever invoked, so accidental deletions are prevented at the client layer as well.

```
2  require_once 'auth.php';
3  require_once '../config/db.php';
4
5  if (!isset($_GET['id']) || !is_numeric($_GET['id'])) {
6      header("Location: dashboard.php");
7      exit();
8  }
9
10 $id = $_GET['id'];
11
12 $stmt = $conn->prepare("SELECT * FROM regions WHERE id = ?");
13 $stmt->execute([$id]);
14 $region = $stmt->fetch(PDO::FETCH_ASSOC);
15
16 if ($region) {
17     if (!empty($region['main_image']) && file_exists('../uploads/images/' . $region['main_image'])) {
18         unlink('../uploads/images/' . $region['main_image']);
19     }
20     if (!empty($region['gallery_image1']) && file_exists('../uploads/images/' . $region['gallery_image1'])) {
21         unlink('../uploads/images/' . $region['gallery_image1']);
22     }
23     if (!empty($region['gallery_image2']) && file_exists('../uploads/images/' . $region['gallery_image2'])) {
24         unlink('../uploads/images/' . $region['gallery_image2']);
25     }
26     if (!empty($region['gallery_image3']) && file_exists('../uploads/images/' . $region['gallery_image3'])) {
27         unlink('../uploads/images/' . $region['gallery_image3']);
28     }
29
30     $stmt = $conn->prepare("DELETE FROM regions WHERE id = ?");
31     $stmt->execute([$id]);
32
33     $_SESSION['message'] = 'تم حذف السجل بنجاح';
34     $_SESSION['msg_type'] = 'success';
35 } else {
36     $_SESSION['message'] = 'السجل غير موجود';
37     $_SESSION['msg_type'] = 'error';
38 }
```

Figure 4.5: `delete.php`, removing the associated image files before deleting the database record.

5. Frontend Screenshots

Illustrating various pages designs such as landing page, gallery page, admin page and place details page. All pages are supported with a **night (dark) mode** design, **responsiveness** across various screen sizes and **well aligned**.

5.1 Home Page (Standard Mode)

The home page introduces the website, its purpose, and the cultural identity of the Kingdom. The top navigation bar links to all public pages and includes a night-mode toggle button (shown here as a crescent moon, indicating that clicking it would switch the site into night mode). Below the introductory text, an "About" section presents the site's key themes.

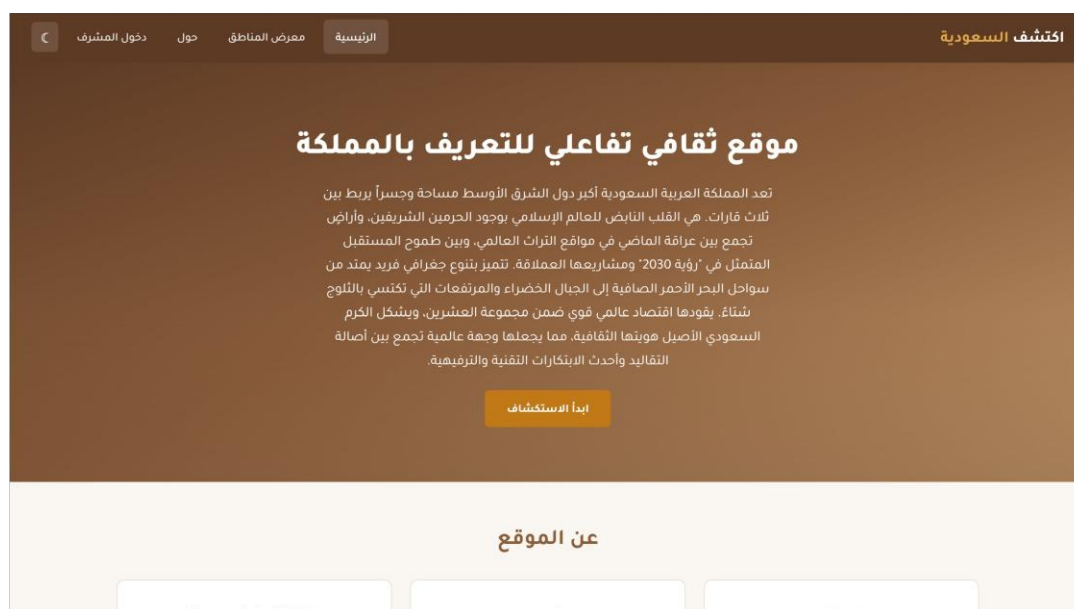


Figure 5.1: Home page in standard (light) mode.

5.2 Home Page (Night Mode Activated)

Night mode is implemented in pure JavaScript by toggling a CSS class on the <body> element. The user's preference is persisted via localStorage so the chosen theme survives page navigation and refreshes. When night mode is active the toggle icon flips to a sun, indicating that clicking it would return the site to standard mode.

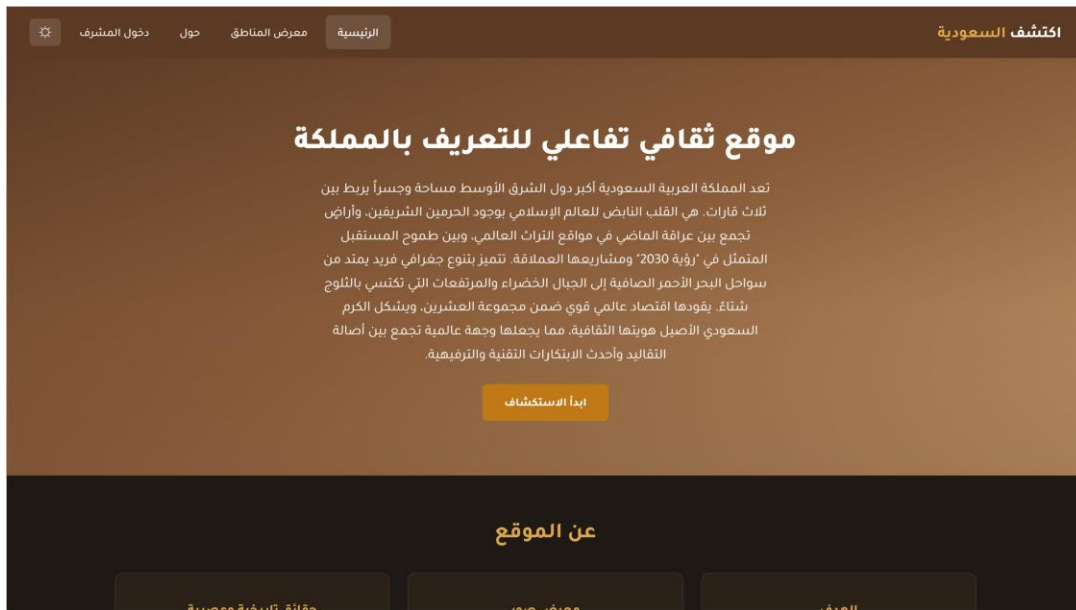


Figure 5.2: Home page after the night-mode button has been activated.

5.3 Regions Gallery Page

The gallery page displays every region record from the database as a card. A free-text search field and a category drop-down filter the visible cards in real time using JavaScript, and a live counter ("عدد النتائج") reports how many results currently match the active filters. Clicking a card navigates the user to the corresponding details page.

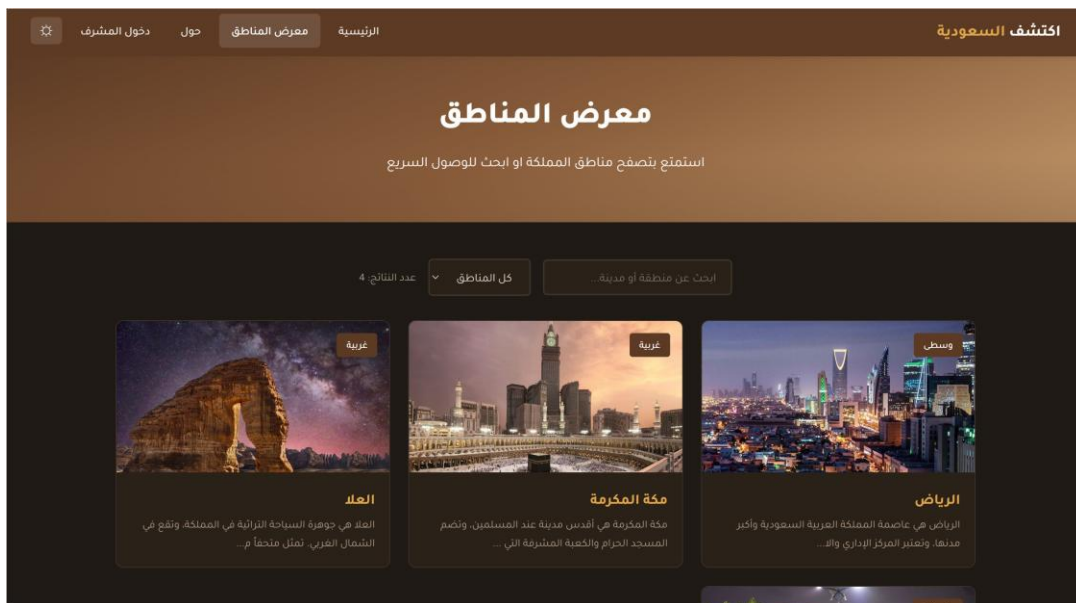


Figure 5.3: The regions gallery, with the search field, category drop-down, and results counter visible.

5.4 Region / Place Details Page

Selecting a card on the gallery page opens its details page, which fetches the chosen record from the database by id and renders its full content: hero image, narrative description, a list of facts, the most prominent landmarks, climate information, and a small image gallery. The screenshot below shows the entire page from top to bottom.

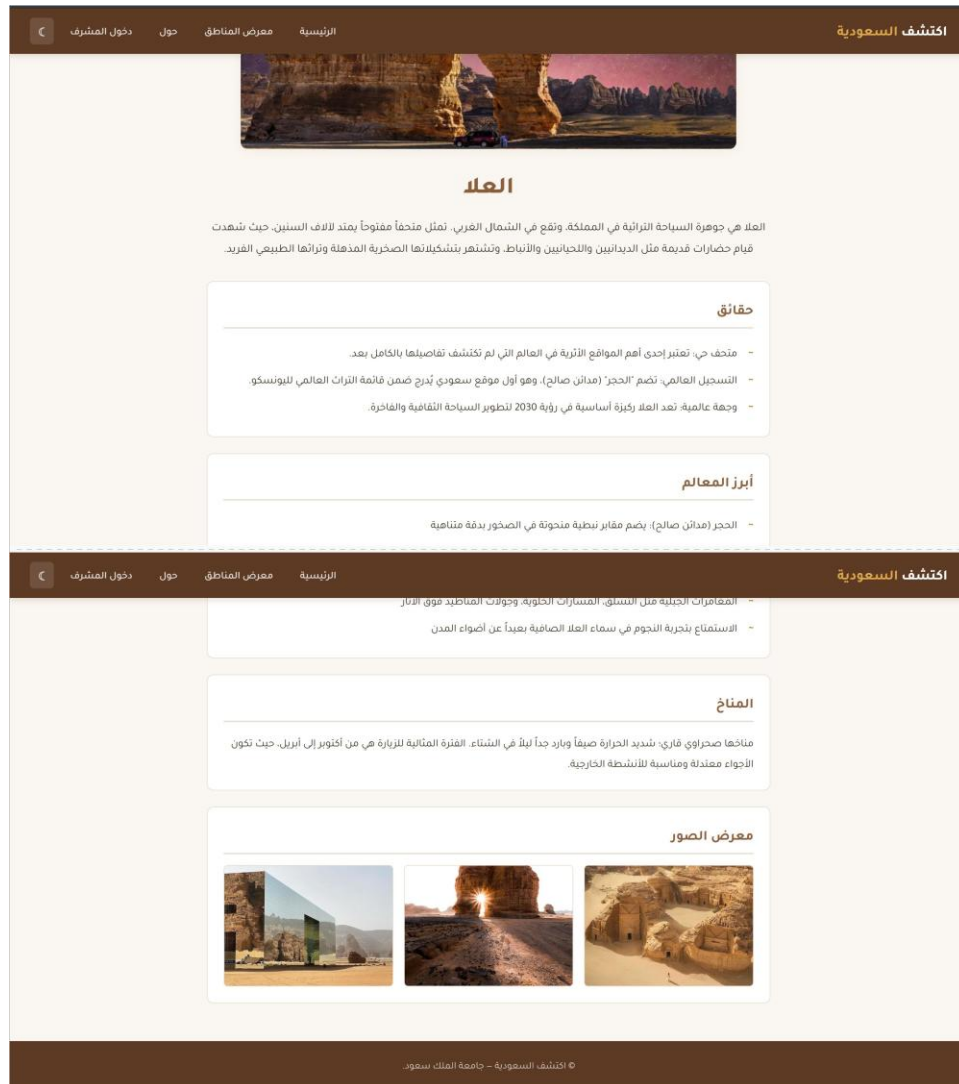


Figure 5.4: The full details page for a single region, scrolled from top to bottom.

5.5 Admin Login Page (with Validation)

The login form includes both client-side JavaScript validation (empty fields, minimum password length) and server-side validation against the admins table. The screenshot below captures a failed login attempt: the user submitted an unrecognised username, and the server-side check has rendered a clear Arabic error message inside a soft pink alert box above the form.

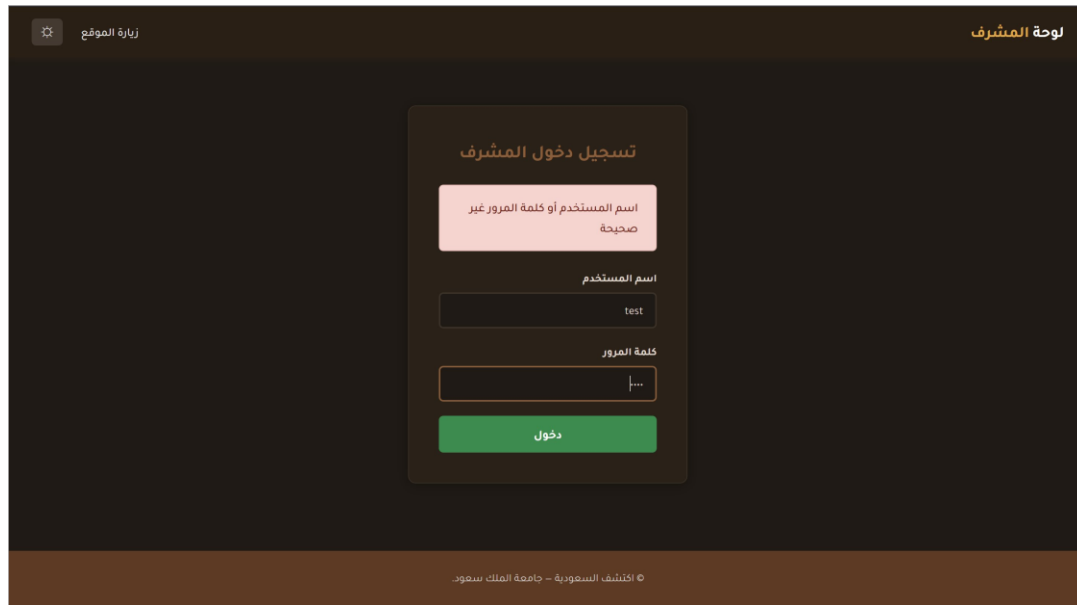


Figure 5.5: The admin login page, showing form validation in action after a failed attempt.

5.6 Admin Dashboard

The dashboard lists every region record in a clean table alongside three actions per row (Preview, Edit, Delete), plus an "Add New Content" button at the top. Successful CRUD operations are confirmed via a flash-style banner shown on this same page, as required by the project specification.

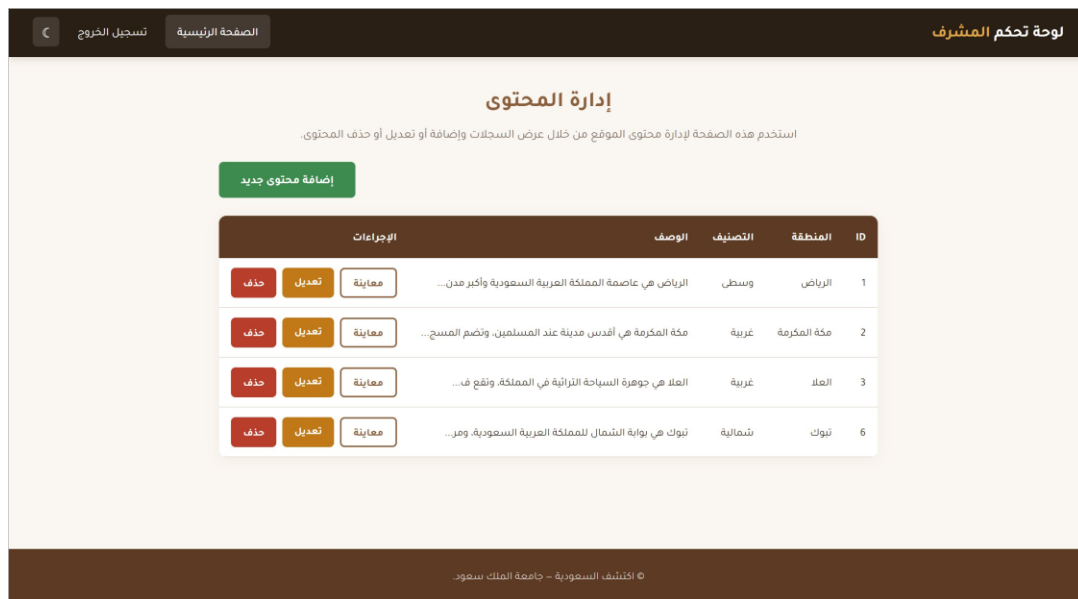


Figure 5.6: The admin dashboard, listing the region records and their CRUD actions.

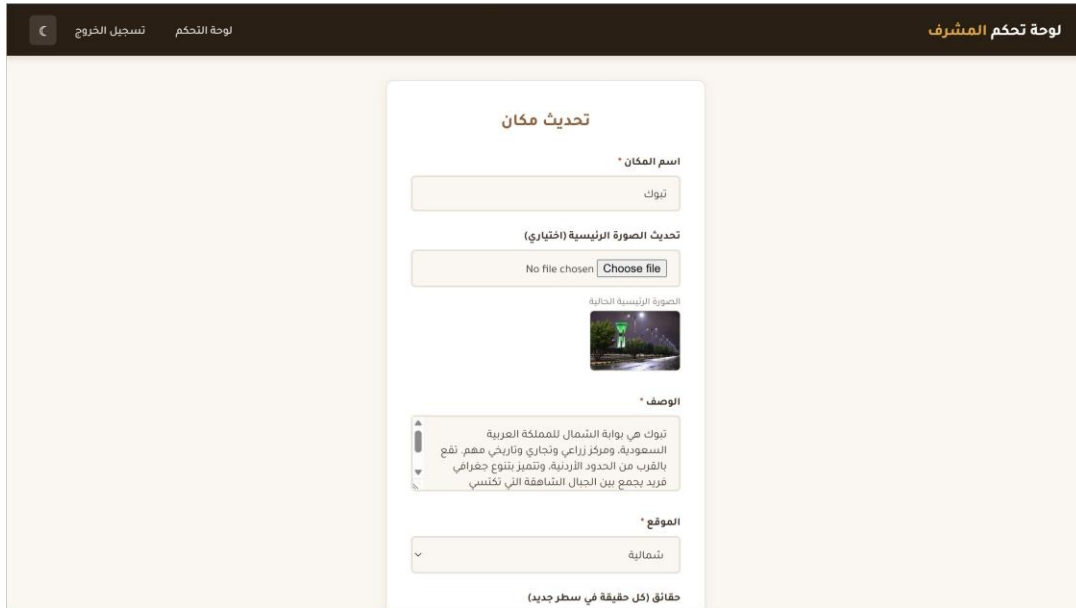
5.7 Add Content Page

The add-content form is used to create a new region record. Required fields are clearly marked with a red asterisk. The form supports a main image upload and accepts free-form text for the description, facts, climate, activities, and landmarks of the new region.

Figure 5.7: The Add Content form for inserting a new region.

5.8 Update Content Page

The update form is pre-filled with the current data of the selected record. The current main image is displayed as a thumbnail under the file picker, so the admin can see what is already saved before deciding whether to upload a replacement; if no new file is selected the existing image is preserved.



لوحة تحكم المشرف

تسجيل الخروج لوحة التحكم

تحديث مكان

اسم المكان *

تيوك

تحديث الصورة الرئيسية (اختياري)

No file chosen Choose file

الصورة الرئيسية الحالية

الوصف *

تيوك هي بوابة الشمال للمملكة العربية السعودية، ومركز زراعي وتجاري وتاريخي مهم. تقع بالقرب من الحدود الأردنية، وتتميز بتنوع جغرافي فريد يجمع بين الجبال الشاهقة التي تكتسي

الموقع *

شمالية

حفظ (كل حقل في سطر جديد)

Figure 5.8: The Update Content form, pre-filled with the selected record's existing data.

End of Report